

UNINASSAU

Fábrica de Software e Tópicos Avançados em Ciência da Computação
Turma 8NA | Semestre 2026.2

RotaCerta

Documento Técnico Cumulativo das Sprints 01 a 03

Grupo 04

Álvaro Jordão | 01748200

Arthur Sales | 01593811

Vinícius Trigueiro | 01794959

William Coelho de Moraes | 01263977

2026

Apresentação

Este documento reúne as entregas das Sprints 01, 02 e 03 do projeto RotaCerta. A Parte I registra o problema, os objetivos, os requisitos e o planejamento inicial. A Parte II apresenta a arquitetura, os modelos de software e de dados, os protótipos e a estrutura inicial do banco. A Parte III demonstra a estrutura funcionando: banco conectado, login, cadastro de usuários, controle de perfis e o CRUD de pedidos, com evidências capturadas do sistema em execução.

As Partes I e II mantêm o conteúdo entregue anteriormente, incluindo as revisões feitas após as orientações da Sprint 01. Os ajustes de planejamento decorrentes da Sprint 03 estão registrados e justificados na seção 22.

Sumário

Parte I Sprint 01 Planejamento do projeto

- 1 Identificação da equipe
- 2 Tema, problema, objetivos e público alvo
- 3 Requisitos funcionais e não funcionais
- 4 Casos de uso e Product Backlog
- 5 Cronograma semanal revisado

Parte II Sprint 02 Arquitetura e modelagem

- 6 Arquitetura do sistema
- 7 Diagrama de classes
- 8 Modelo Entidade Relacionamento
- 9 Modelo relacional
- 10 Protótipos das telas
- 11 Banco de dados e projeto no GitHub
- 12 Avaliação sequencial e paralela
- 13 Aderência às orientações da disciplina

Parte III Sprint 03 Estrutura inicial funcionando

- 14 Estrutura implementada
- 15 Banco de dados conectado
- 16 Login funcional
- 17 Cadastro de usuários
- 18 Controle de perfis
- 19 CRUD principal: pedidos
- 20 Execução local
- 21 Testes automatizados
- 22 Ajustes no planejamento e na modelagem
- 23 Repositório GitHub
- 24 Dificuldades encontradas e próximos passos

Parte I Sprint 01 Planejamento do projeto

1 Identificação da equipe

Integrante	Matrícula	Responsabilidade inicial
Álvaro Jordão	01748200	Scrum Master e Product Owner; organização, requisitos e priorização
Arthur Sales	01593811	Backend e núcleo de otimização e paralelização
Vinícius Trigueiro	01794959	Frontend, banco de dados e documentação
William Coelho de Moraes	01263977	Desenvolvimento e testes

Projeto RotaCerta. Turma 8NA. As responsabilidades organizam o trabalho, mas todos os integrantes participam das decisões, implementação, testes e documentação.

2 Tema, problema, objetivos e público alvo

Tema. Logística e otimização de processos para pequenos negócios com operação própria de entrega.

Problema. Farmácias, mercados, restaurantes e pequenas distribuidoras frequentemente planejam entregas de forma manual. Essa prática aumenta a distância percorrida, o consumo de combustível e a possibilidade de atraso quando o volume de pedidos cresce.

Objetivo geral. Desenvolver um sistema web que distribua pedidos entre entregadores e determine uma sequência de paradas que reduza a distância e o tempo total das rotas.

Resultados esperados. Centralizar pedidos e entregadores, reduzir distância e tempo em relação à alocação manual, calcular rotas para vários entregadores e disponibilizar uma interface responsiva para a operação diária.

Público alvo. Administradores, atendentes e entregadores de pequenos negócios locais. Os clientes finais são beneficiados por entregas mais previsíveis.

3 Requisitos funcionais e não funcionais

Requisitos funcionais

ID	Descrição
RF01	Cadastrar e autenticar usuários com perfis de administrador, atendente e entregador.
RF02	Cadastrar pedidos com cliente, endereço, prioridade e horário desejado.
RF03	Cadastrar entregadores com disponibilidade e capacidade de carga.
RF04	Gerar automaticamente uma rota otimizada para cada entregador.
RF05	Atribuir pedidos a entregadores de forma manual ou automática.
RF06	Exibir a rota como lista ordenada de paradas e mapa.
RF07	Atualizar o status do pedido entre pendente, atribuído, em rota e entregue.

ID	Descrição
RF08	Manter o histórico de entregas realizadas.
RF09	Gerar relatório com tempo, distância e comparação da otimização.
RF10	Disponibilizar painel administrativo da operação do dia.

Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Desempenho	Executar o cálculo em tempo aceitável para vários entregadores e permitir processamento paralelo.
RNF02	Segurança	Autenticar usuários, autorizar por perfil e armazenar senhas com hash seguro.
RNF03	Usabilidade	Manter a interface simples para usuários com diferentes níveis de letramento técnico.
RNF04	Portabilidade	Funcionar em navegadores e adaptar o layout para celulares.
RNF05	Escalabilidade	Suportar o crescimento de pedidos e entregadores sem degradação significativa.
RNF06	Confiabilidade	Preservar pedidos, rotas e histórico com integridade referencial.
RNF07	Manutenibilidade	Manter código organizado, documentado, testável e versionado.

4 Casos de uso e Product Backlog

ID	Ator	Caso de uso
UC01	Atendente ou administrador	Cadastrar pedido
UC02	Administrador	Cadastrar entregador
UC03	Administrador	Gerar rotas para os pedidos do dia
UC04	Entregador	Visualizar a rota atribuída
UC05	Entregador	Atualizar o status da entrega
UC06	Administrador	Consultar métricas das rotas
UC07	Todos os perfis	Autenticar-se no sistema

Product Backlog inicial

Alta prioridade. Autenticação e perfis, CRUD de pedidos, CRUD de entregadores, banco de dados e motor sequencial com vizinho mais próximo e 2-opt.

Média prioridade. Visualização das rotas, atualização de status e paralelização para múltiplos entregadores.

Evolução. Relatórios, painel consolidado, notificações e avaliação futura de C++ com OpenMP ou CUDA.

5 Cronograma semanal revisado

O cronograma abaixo substituiu, na Sprint 02, a periodicidade quinzenal apresentada inicialmente. A Sprint 02 manteve o prazo excepcional de 19 de setembro comunicado pela professora; as etapas seguintes usam ciclos semanais. A versão vigente, ajustada na Sprint 03, está na seção 22.

Sprint	Período	Entrega principal
1	03/09 a 05/09	Planejamento, requisitos e backlog
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura do projeto
3	20/09 a 26/09	Ambiente executável e contrato inicial da API
4	27/09 a 03/10	Autenticação e autorização por perfil
5	04/10 a 10/10	CRUD de clientes e pedidos
6	11/10 a 17/10	CRUD de entregadores e disponibilidade
7	18/10 a 24/10	Atribuição de pedidos e visualização de rotas
8	25/10 a 31/10	Vizinho mais próximo em modo sequencial
9	01/11 a 07/11	2-opt e linha de base de desempenho
10	08/11 a 14/11	Paralelização para múltiplos entregadores
11	15/11 a 21/11	Experimentos sequencial versus paralelo
12	22/11 a 28/11	Relatórios, testes e documentação
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca

Parte II Sprint 02 Arquitetura e modelagem

A Sprint 02 transforma os requisitos em uma base técnica verificável. A solução adota uma arquitetura web modular, um modelo de dados relacional e um componente de otimização isolado, permitindo implementar e medir versões sequencial e paralela sem duplicar regras de negócio.

6 Arquitetura do sistema

Camada	Tecnologia	Responsabilidade
Interface	React, TypeScript e Vite	Aplicação responsiva para administração e operação móvel
API	FastAPI e Python 3.12	Endpoints REST, validação, OpenAPI e integração com o otimizador
Persistência	PostgreSQL 16	Integridade, índices, histórico e métricas de execução
Mapa	Leaflet, OpenStreetMap e adaptador OSRM	Exibição de rotas e matriz de distâncias sem prender o domínio a um provedor
Otimização	Python, 2-opt e processos independentes	Linha de base sequencial e cálculo paralelo por entregador
Ambiente	Docker Compose	Inicialização reproduzível de frontend, API e banco

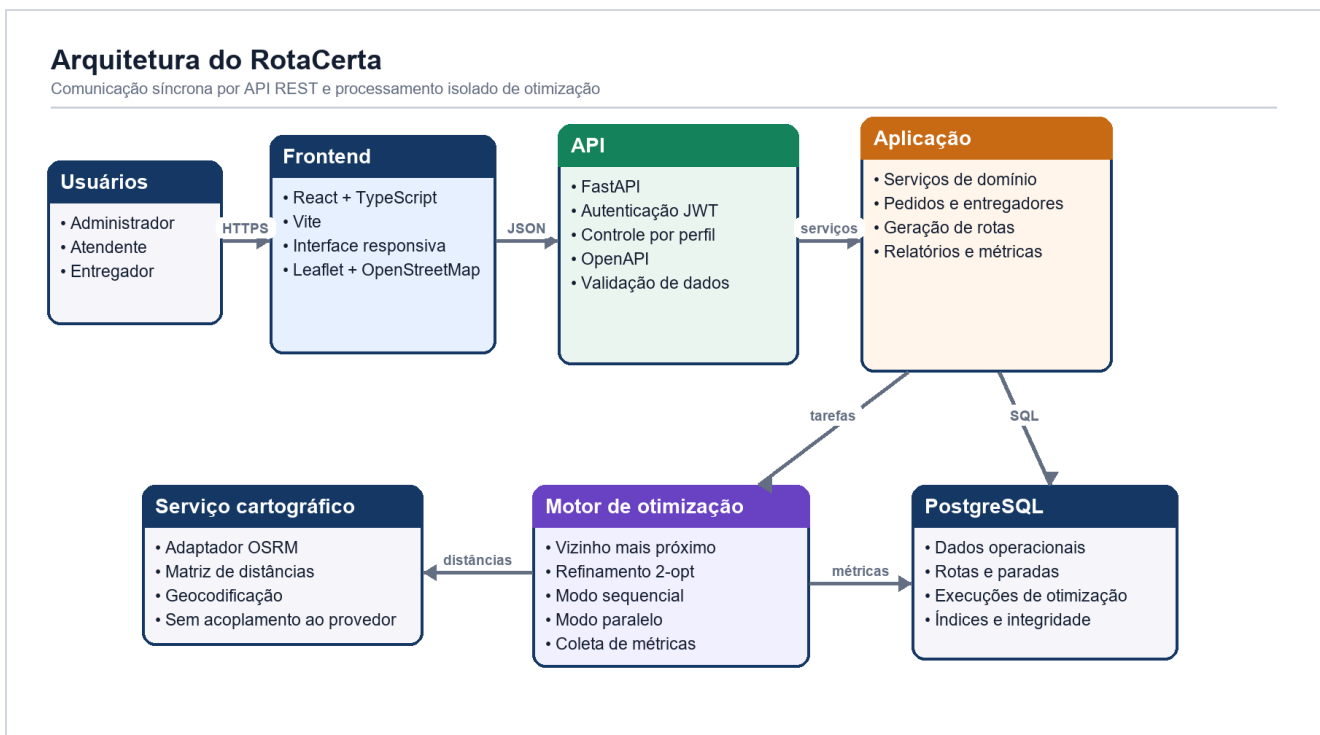


Figura 1 Arquitetura lógica e comunicação entre os componentes

O navegador envia requisições HTTPS em JSON para a API. A API executa regras de negócio, persiste dados no PostgreSQL e aciona o motor de otimização. O motor recebe pedidos, entregadores e uma matriz de distâncias por meio de uma interface estável. A mesma interface atende aos modos sequencial e paralelo, o que permite comparar desempenho sem alterar os dados de entrada.

A escolha de Python não pressupõe o uso de inteligência artificial. As orientações da disciplina aceitam inteligência artificial e/ou otimização de processamento, paralelização e alto desempenho. No RotaCerta,

o componente avançado é a otimização de rotas integrada ao fluxo principal. React e TypeScript permanecem restritos à interface, enquanto Python executa o núcleo computacional.

7 Diagrama de classes

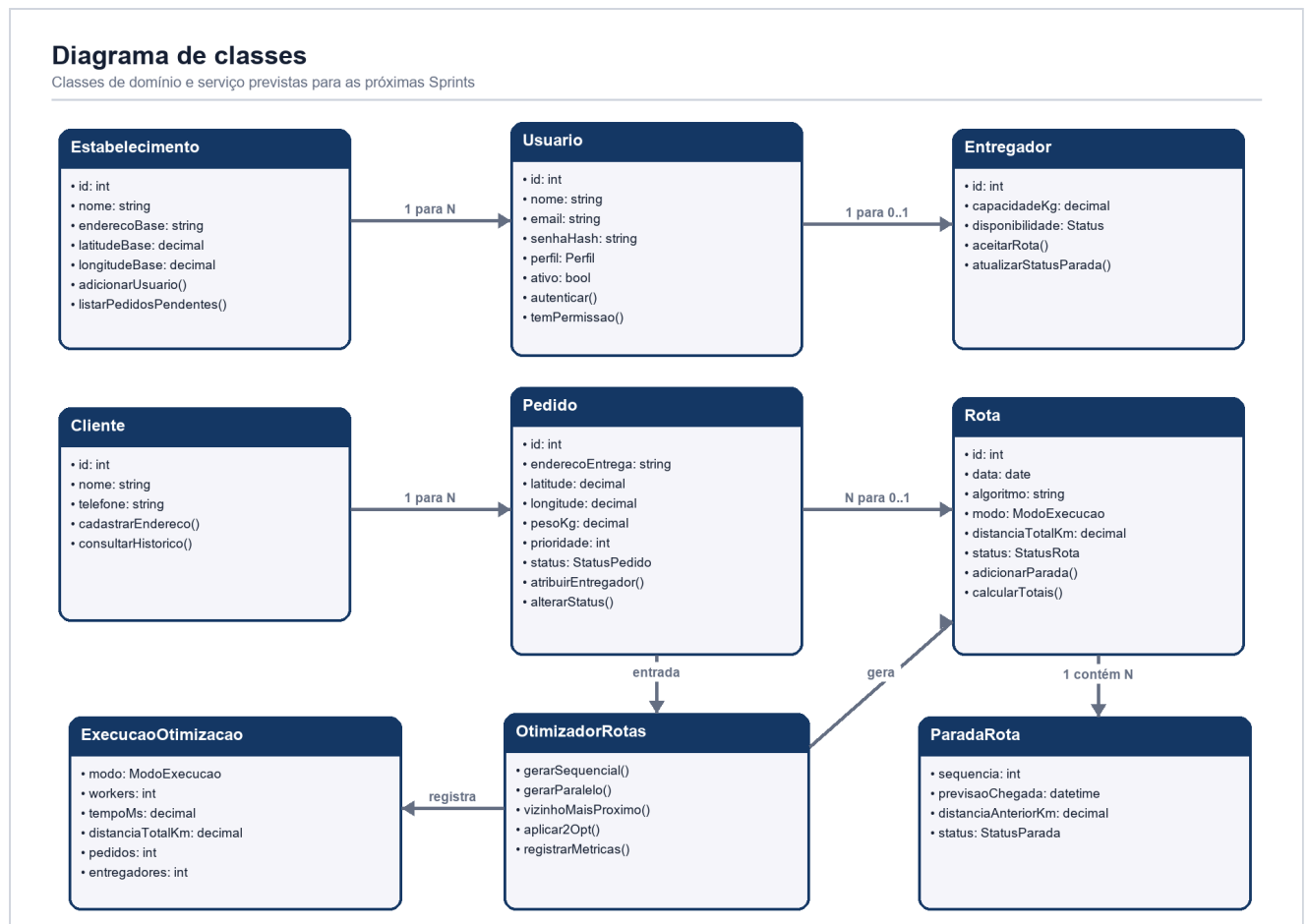


Figura 2 Classes de domínio e serviço previstas

Estabelecimento delimita os dados de cada negócio. Usuario representa autenticação e perfil; Entregador complementa um usuário operacional com capacidade e disponibilidade. Pedido registra destino, prioridade, janela de horário e status. Rota agrega paradas ordenadas. OtimizadorRotas executa o mesmo algoritmo nos modos sequencial e paralelo e registra cada ExecucaoOtimizacao.

8 Modelo Entidade Relacionamento

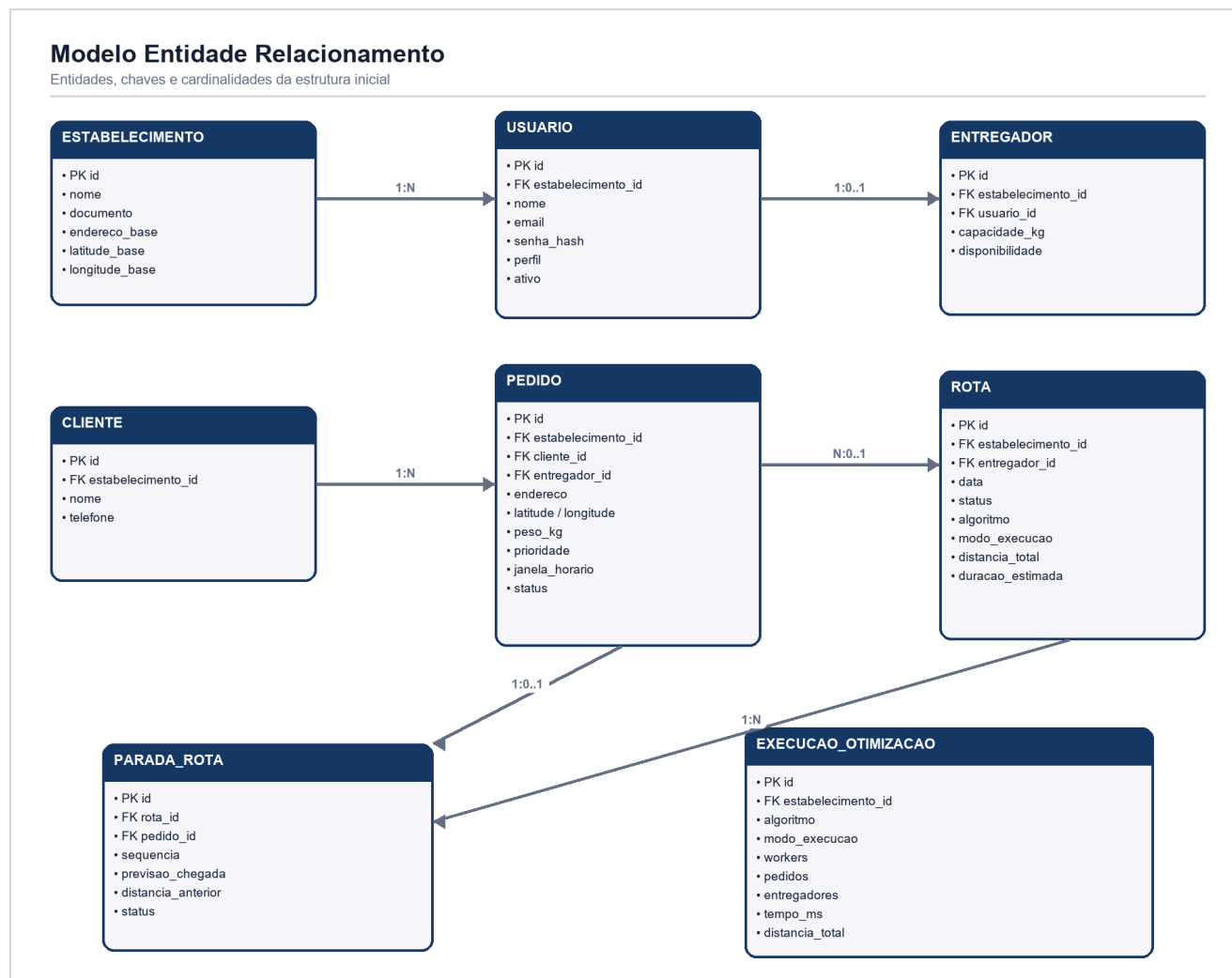


Figura 3 Modelo conceitual com chaves e cardinalidades

O modelo mantém o estabelecimento como limite organizacional. Um usuário pode possuir um cadastro de entregador. Cada pedido pertence a um cliente e pode ser atribuído a um entregador. Uma rota contém paradas ordenadas; cada parada referencia um pedido. As execuções de otimização registram quantidade de pedidos, entregadores, workers, tempo e distância total.

9 Modelo relacional

Tabela	PK	FK	Finalidade
establishments	id	-	Negócio, endereço e coordenadas do depósito
users	id	establishment_id	Credenciais, perfil e situação do usuário
couriers	id	establishment_id, user_id	Capacidade e disponibilidade do entregador
customers	id	establishment_id	Cliente associado ao estabelecimento
orders	id	establishment_id, customer_id, assigned_courier_id	Destino, peso, prioridade, janela e status
routes	id	establishment_id, courier_id	Rota diária, algoritmo, modo e totais
route_stops	id	route_id, order_id	Sequência e acompanhamento de cada parada

Tabela	PK	FK	Finalidade
optimization_runs	id	establishment_id	Métricas de execuções sequenciais e paralelas

As restrições CHECK limitam perfis, estados e modos de execução. As chaves estrangeiras preservam a integridade entre usuários, pedidos, entregadores e rotas. Índices cobrem as consultas mais frequentes por estabelecimento, status, entregador, data e modo de execução.

10 Protótipos das telas principais

Os wireframes priorizam navegação direta e leitura em dispositivos móveis. A primeira sequência cobre autenticação, painel diário e cadastro de pedido. A segunda cobre geração de rotas, consulta da rota pelo entregador e atualização de status.

Protótipos principais

Fluxo administrativo em baixa fidelidade

RotaCerta

Acesso

Entrar no sistema

E-mail

Senha

Entrar

Perfis: administrador, atendente e entregador

RotaCerta

Painel do dia

Pedidos
38

Em rota
16

Entregues
19

Operação

#1038 Ana Souza Pendente

#1037 João Lima Em rota

#1036 Carla Melo Entregue

Novo pedido

Gerar rotas

RotaCerta

Novo pedido

Cliente

Endereço

Peso em kg
2,50

Prioridade
Normal

Horário desejado

Cancelar

Salvar pedido

Figura 4 Acesso, painel administrativo e cadastro de pedido

Protótipos principais

Geração de rotas e operação móvel do entregador

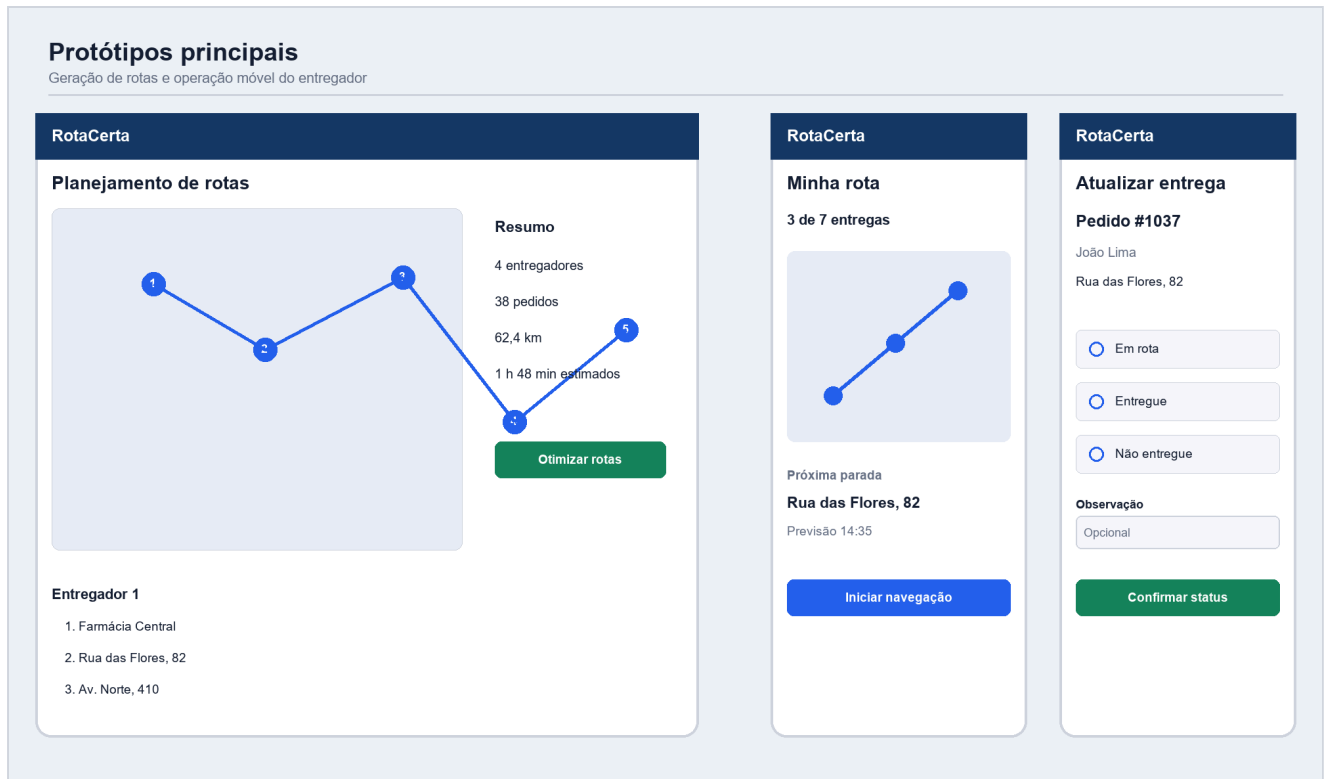


Figura 5 Planejamento de rotas e fluxo móvel do entregador

Tela	Requisito	Objetivo
Acesso	UC07	Login e identificação do perfil
Painel do dia	RF10	Pedidos, situação operacional e atalhos
Novo pedido	UC01 / RF02	Cliente, destino, carga, prioridade e janela
Planejamento	UC03 / RF04	Mapa, totais e comando de otimização
Minha rota	UC04 / RF06	Sequência, mapa e próxima parada
Atualizar entrega	UC05 / RF07	Transição de status e observação

11 Banco de dados e projeto no GitHub

Banco de dados criado

A estrutura inicial está implementada no arquivo database/schema.sql para PostgreSQL 16. O serviço db do arquivo compose.yaml executa o esquema automaticamente na primeira inicialização do volume. O modelo inclui oito tabelas, restrições de integridade e cinco índices iniciais.

Estrutura do repositório na Sprint 02

Caminho	Conteúdo
backend/	API FastAPI e limite do módulo de otimização
backend/app/optimizer/	Vizinho mais próximo, 2-opt e execução sequencial ou paralela
backend/tests/	Testes automatizados do núcleo de roteirização
frontend/	Aplicação React e TypeScript
database/	DDL, instruções e inicialização do PostgreSQL
docs/sprint-02/assets/	Diagramas e protótipos incorporados ao documento
tools/documentation/	Geração reproduzível dos materiais técnicos
compose.yaml	Orquestração local dos três serviços
CONTRIBUTING.md	Padrões de branch, commit e revisão

O fluxo recomendado usa branches curtas por tarefa, pull request e revisão de outro integrante. As branches pessoais dev/alvaro, dev/arthur e dev/vinicius identificam ambientes acadêmicos, mas não substituem branches de feature, correção ou documentação.

12 Avaliação sequencial e paralela

A orientação da Sprint 01 exige manter a comparação entre as versões sequencial e paralela. A primeira implementação já executa vizinho mais próximo e refinamento 2-opt nos dois modos. As versões usam os mesmos pedidos, entregadores e coordenadas; a única variável controlada é o modo de execução.

Métrica	Cálculo ou unidade	Critério
Tempo de execução	Milissegundos	Média, mediana e percentil 95 de dez repetições
Distância total	Quilômetros	Confirma que a paralelização não altera a qualidade da rota
Speedup	T_{seq} / T_{par}	Ganho obtido com a execução paralela
Eficiência	$Speedup / workers$	Uso relativo dos processos disponíveis
Escala	Pedidos e entregadores	Cenários de 50, 100, 250 e 500 pedidos com 2, 4 e 8 entregadores

Cada cenário terá uma execução de aquecimento e dez repetições válidas. O relatório registrará processador, memória, sistema operacional, versão do código, número de workers e semente dos dados. A tabela optimization_runs armazenará cada medição para permitir auditoria e reprodução.

Evidência prática. O arquivo backend/app/optimizer/routing.py contém a heurística determinística, o cálculo de distância geodésica, o refinamento 2-opt e a execução com processos independentes. O endpoint POST /optimizer/compare retorna rotas, distâncias, tempos, workers, speedup e a confirmação de equivalência entre os modos. Cinco testes automatizados validam rota vazia, preservação das paradas, melhoria do 2-opt, resultado da otimização e equivalência sequencial/paralela.

13 Aderência às orientações da disciplina

Orientação	Situação na Sprint 02	Evidência
Projeto integrado	Atendido	Um único sistema reúne interface, API, banco e otimização
Componente avançado	Atendido	Otimização e paralelização fazem parte da geração de rotas
Tecnologia prioritária	Atendido	Python no núcleo; JavaScript e TypeScript apenas na interface
Integração não superficial	Atendido	O endpoint compara os modos sobre os mesmos dados
Evolução prática	Atendido	Código executável e cinco testes automatizados no repositório
GitHub atualizado	Atendido	Branch da Sprint 02 e Pull Request número 1
Participação da equipe	Contínuo	Cada integrante deve registrar contribuições próprias nas próximas tarefas

Parte III Sprint 03 Estrutura inicial funcionando

A Sprint 03 transforma a arquitetura e o modelo de dados da Sprint 02 em um sistema executável. Frontend, API e PostgreSQL passaram a funcionar integrados: o usuário faz login, recebe permissões de acordo com o seu perfil e cadastra, consulta, edita e exclui pedidos gravados no banco. Todas as telas e respostas mostradas nesta parte foram capturadas com o sistema em execução local.

14 Estrutura implementada

A aplicação mantém as três camadas definidas na Sprint 02. O frontend React consome a API REST com requisições JSON autenticadas por token. A API valida os dados, aplica as regras de cada perfil e grava as informações no PostgreSQL por meio do SQLAlchemy.

Componente	Arquivo	Responsabilidade
Configuração	backend/app/config.py	Lê a URL do banco, a chave do token, as origens permitidas e as opções de inicialização
Conexão	backend/app/database.py	Cria o pool de conexões, verifica o banco e aplica o esquema quando necessário
Modelos	backend/app/models.py	Mapeia as tabelas do schema.sql para classes Python
Segurança	backend/app/security.py	Hash Argon2, emissão e validação do JWT e verificação de perfil
Autenticação	backend/app/routers/auth.py	Login, cadastro de estabelecimento e dados do usuário logado
Usuários	backend/app/routers/users.py	Cadastro, edição e desativação de usuários; lista de entregadores
Pedidos	backend/app/routers/orders.py	CRUD de pedidos e mudança de situação
Dados iniciais	backend/app/seed.py	Estabelecimento, usuários e pedidos de demonstração
Interface	frontend/src/pages/	Login, pedidos, entregas do entregador e usuários
Testes	backend/tests/	16 testes da API e 5 do otimizador

Fluxo de uma requisição autenticada

1. O usuário informa e-mail e senha na tela de login.
2. A API compara a senha com o hash Argon2 gravado em `users.password_hash` e devolve um token JWT válido por oito horas.
3. O frontend envia o token no cabeçalho `Authorization` de cada requisição.
4. A API valida o token, carrega o usuário do banco e confere se o perfil tem permissão para a rota.
5. As consultas são sempre filtradas pelo estabelecimento do usuário; entregadores recebem apenas os pedidos atribuídos a eles.

15 Banco de dados conectado

A API se conecta ao PostgreSQL pela URL da variável `DATABASE_URL`, usando o driver `psycopg3` e o `SQLAlchemy 2`. O pool de conexões testa a conexão antes de cada uso. Na execução com Docker, o PostgreSQL aplica `database/schema.sql` na primeira inicialização; na execução sem Docker, a própria

API aplica o mesmo arquivo quando encontra o banco vazio. Nos dois casos a estrutura é exatamente a modelada na Sprint 02.

O endpoint GET /health executa uma consulta no banco e informa o tipo e a versão do servidor. Se o banco estiver indisponível, a resposta passa a ser HTTP 503 com database igual a unavailable.

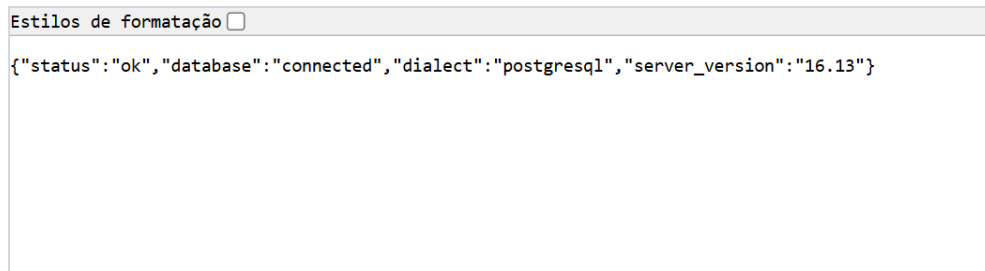


Figura 6 Endpoint de verificação confirmando a conexão da API com o PostgreSQL

```
-- Tabelas do banco rotacerta e quantidade de registros
SELECT 'establishments' AS tabela, count(*) AS registros FROM establishments UNION ALL ... (uma consulta por tabela)
```

tabela	registros
establishments	2
users	6
couriers	3
customers	7
orders	6
routes	0
route_stops	0
optimization_runs	0

(8 linhas)

Consulta às oito tabelas do banco rotacerta com a quantidade de registros

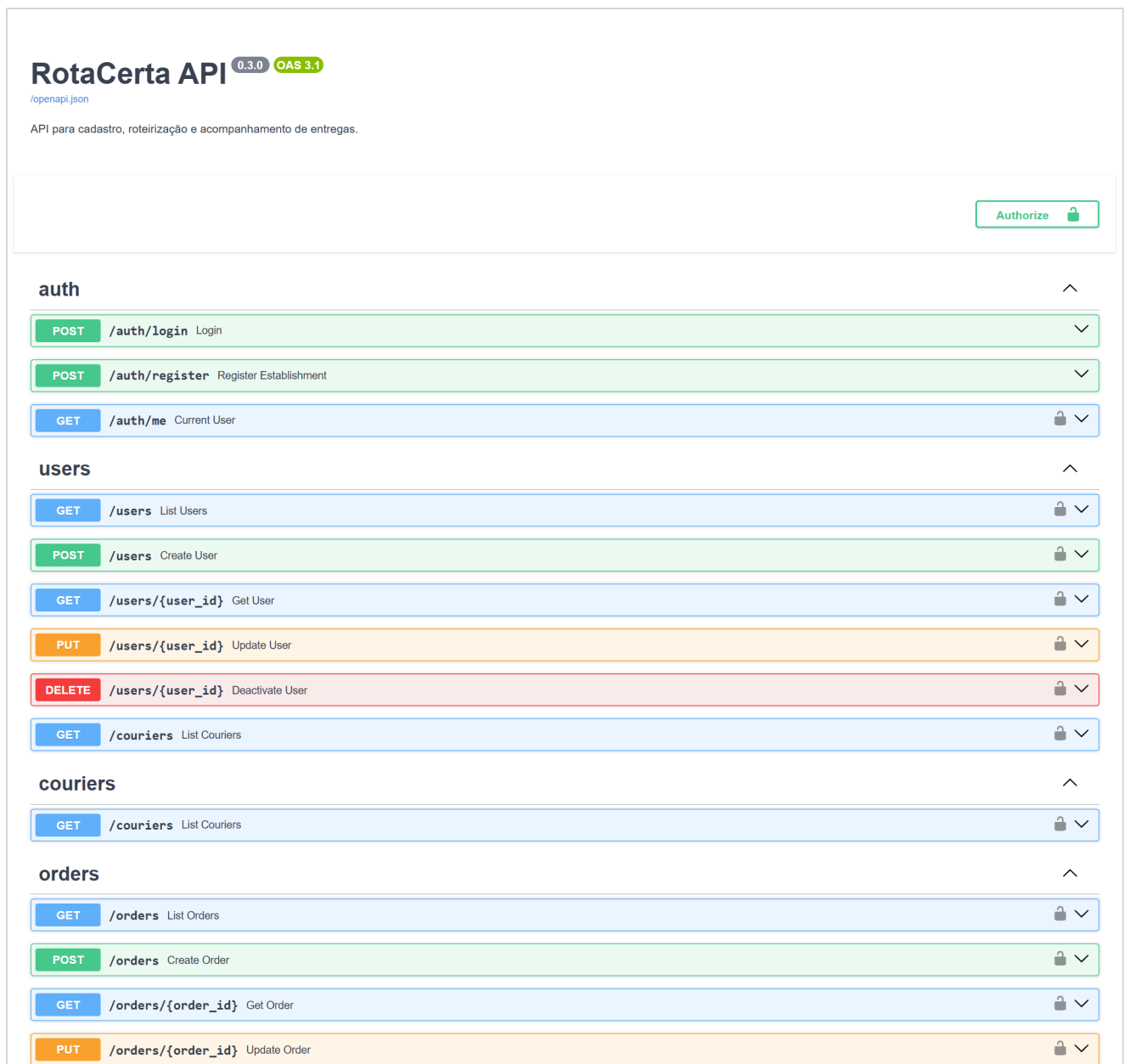


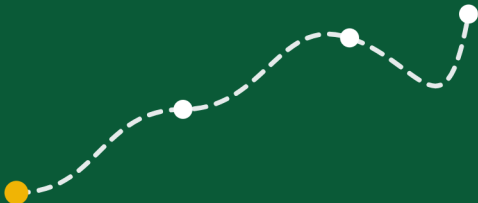
Figura 7 Documentação interativa da API gerada pelo FastAPI, com as rotas de autenticação, usuários e pedidos

16 Login funcional

A tela de login autentica os usuários cadastrados no banco. O e-mail é comparado sem diferenciar maiúsculas de minúsculas e a senha é verificada contra o hash Argon2; a senha em texto nunca é gravada. Credenciais incorretas retornam HTTP 401 com a mensagem E-mail ou senha inválidos. Usuários desativados recebem HTTP 403. A sessão fica salva no navegador e é validada novamente na API sempre que a página é aberta.

RotaCerta

Pedidos, entregadores e rotas do dia em um só lugar.



Entrar

Use o e-mail e a senha cadastrados pelo administrador.

E-mail

Senha

Entrar

Ainda não usa o RotaCerta? [Cadastrar meu negócio](#)

Contas de demonstração, senha rotacerta123

Administrador

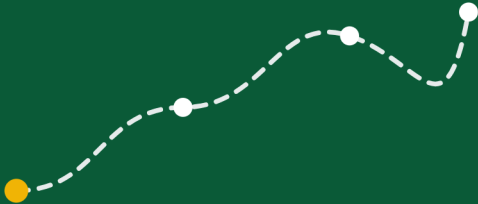
Atendente

Entregador

Figura 8 Tela de login com atalhos para as contas de demonstração

RotaCerta

Pedidos, entregadores e rotas do dia em um só lugar.



Entrar

Use o e-mail e a senha cadastrados pelo administrador.

E-mail ou senha inválidos.

E-mail

Senha

Entrar

Ainda não usa o RotaCerta? [Cadastrar meu negócio](#)

Contas de demonstração, senha rotacerta123

Administrador

Atendente

Entregador

Figura 9 Tentativa de login com senha incorreta

```
-- Usuários cadastrados: a senha fica gravada apenas como hash Argon2
SELECT id, full_name AS nome, email, role AS perfil, active AS ativo, left(password_hash, 32) || '...' AS hash_da_senha
FROM users ORDER BY id
```

id	nome	email	perfil	ativo	hash_da_senha
1	Ana Administradora	admin@rotacerta.com.br	ADMIN	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$3...
2	Bruno Atendente	atendente@rotacerta.com.br	ATTENDANT	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$z...
3	Carla Entregadora	entregador@rotacerta.com.br	COURIER	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$P...
4	Diego Entregador	entregador2@rotacerta.com.br	COURIER	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$D...
5	José Almeida	jose@mercadinhosaojose.com.br	ADMIN	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$S...
6	Eduardo Nascimento	eduardo@rotacerta.com.br	COURIER	True	\$argon2id\$v=19\$m=65536,t=3,p=4\$g...

(6 linhas)

Usuários gravados no banco: apenas o hash da senha é armazenado

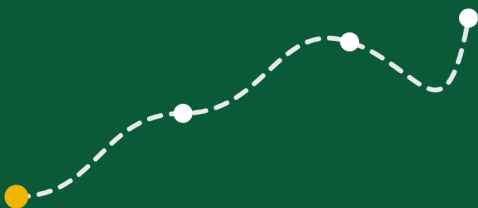
17 Cadastro de usuários

O cadastro acontece de duas formas. Um novo negócio cria a própria conta pela opção Cadastrar meu negócio, que grava o estabelecimento e o seu primeiro administrador. A partir daí, o administrador cadastra atendentes e entregadores na tela Usuários. Para entregadores, o sistema exige a capacidade de carga e cria também o registro na tabela couriers.

- O e-mail é único em todo o sistema; uma duplicidade retorna HTTP 409.
- A senha tem no mínimo oito caracteres e é gravada com hash Argon2.
- Usuários não são apagados: a desativação bloqueia o login e preserva o histórico de pedidos.
- O administrador não pode desativar a si mesmo nem remover o próprio perfil de administrador.
- Cada estabelecimento enxerga apenas os próprios usuários e pedidos.

RotaCerta

Pedidos, entregadores e rotas do dia em um só lugar.



Cadastrar meu negócio

Você será o administrador e poderá cadastrar atendentes e entregadores.

Nome do negócio

Endereço de saída das entregas

Seu nome

E-mail

Senha

Minimo de 8 caracteres.

Criar conta

Já tem conta? [Entrar](#)

Figura 10 Cadastro de um novo estabelecimento com o seu administrador

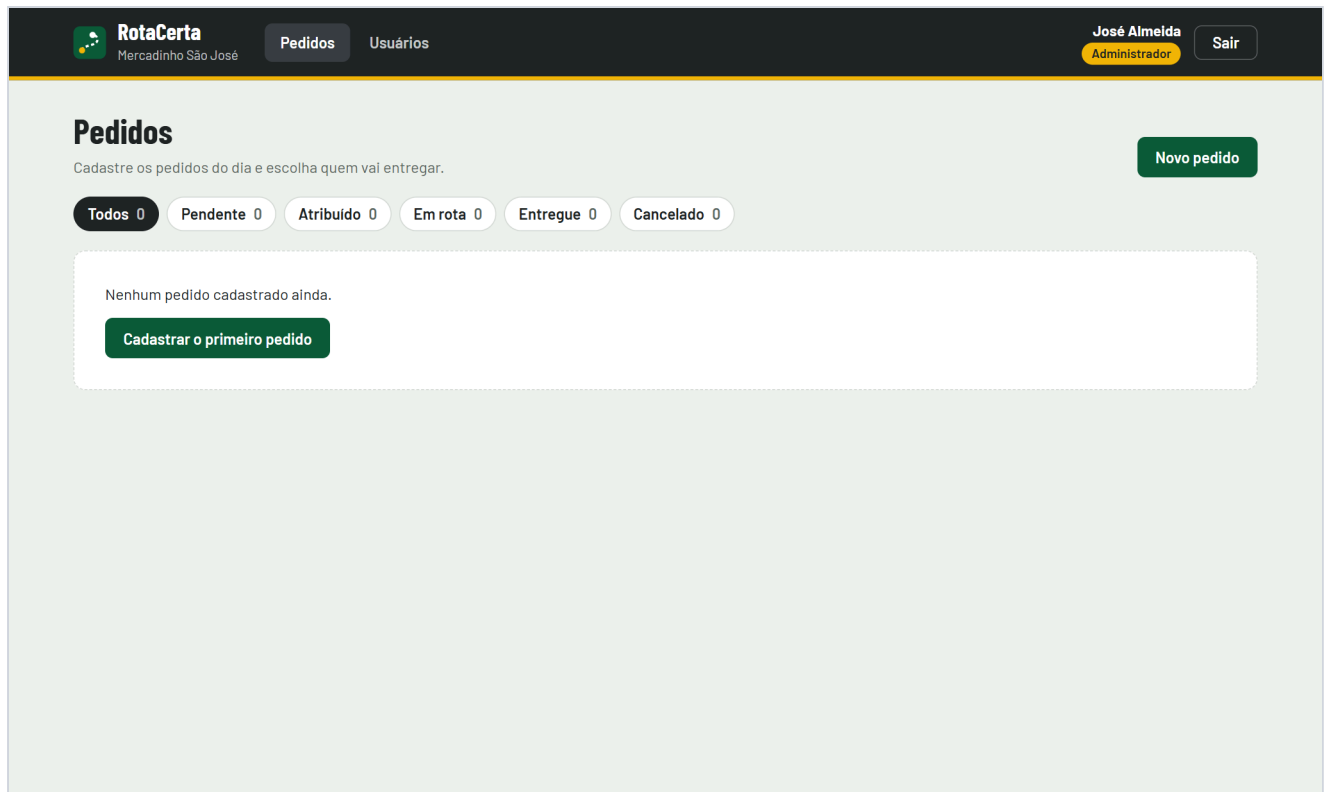


Figura 11 Estabelecimento recém-criado: os pedidos de outros negócios não aparecem

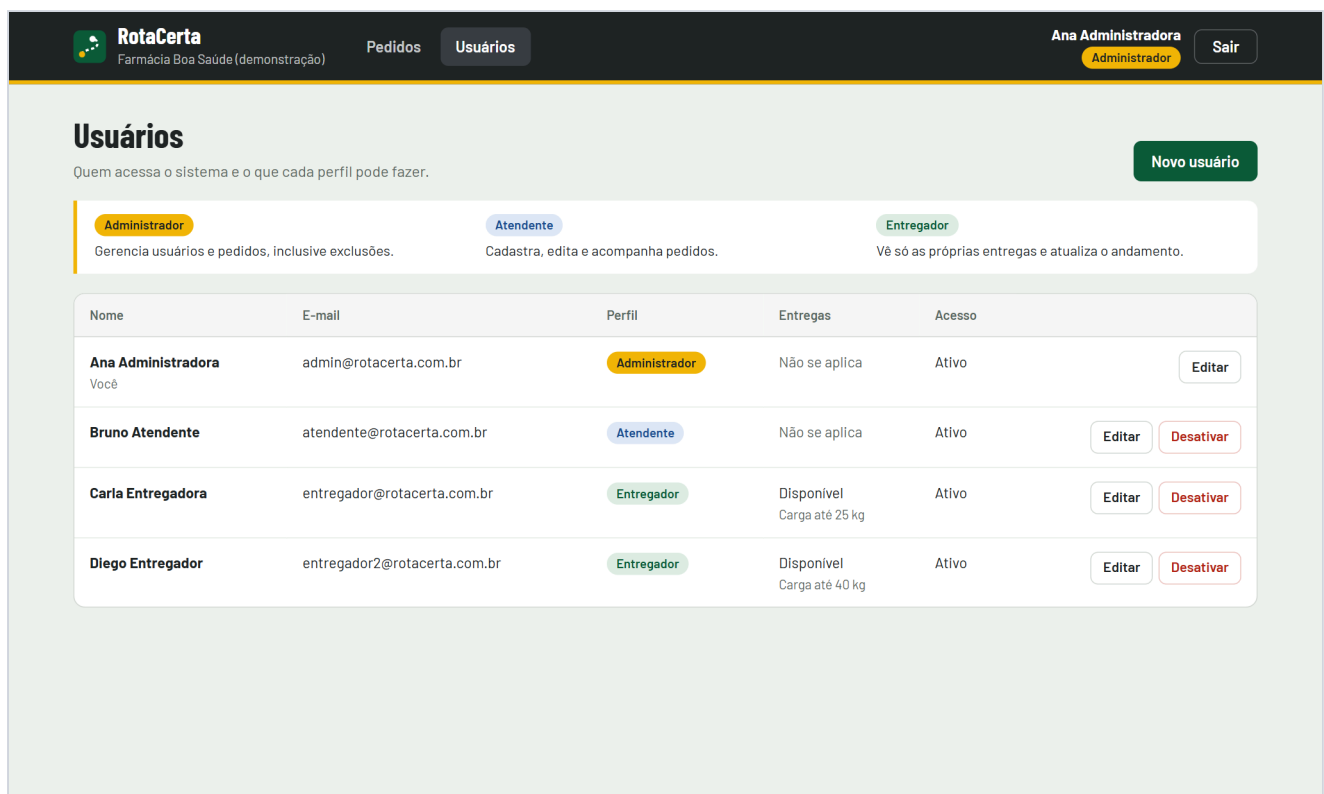


Figura 12 Tela de usuários do administrador, com o resumo das permissões de cada perfil



RotaCerta

Farmácia Boa Saúde (demonstração)

Pedidos

Usuários

Usuários

Quem acessa o sistema e o que cada perfil pode fazer.

Administrador

Gerencia usuários e pedidos, inclusive exclusões.

Atendente

Cadasta, edita e acompanha pedidos.

Nome	E-mail	Perfil
Ana Administradora Você	admin@rotacerta.com.br	Administrador
Bruno Atendente	atendente@rotacerta.com.br	Atendente
Carla Entregadora	entregador@rotacerta.com.br	Entregador
Diego Entregador	entregador2@rotacerta.com.br	Entregador

Novo usuário

Dados de acesso

Nome completo

Eduardo Nascimento

E-mail

eduardo@rotacerta.com.br

Senha

.....

Minimo de 8 caracteres.

Perfil

Administrador

Gerencia usuários e pedidos, inclusive exclusões.

Atendente

Cadasta, edita e acompanha pedidos.

Entregador

Vê só as próprias entregas e atualiza o andamento.

Entregador

Capacidade de carga (kg)

30

Cadastrar usuário

Figura 13 Cadastro de um entregador com capacidade de carga



RotaCerta

Farmácia Boa Saúde (demonstração)

Pedidos

Usuários

Ana Administradora

Administrador

Sair

Usuários

Quem acessa o sistema e o que cada perfil pode fazer.

Administrador

Gerencia usuários e pedidos, inclusive exclusões.

Atendente

Cadasta, edita e acompanha pedidos.

Entregador

Vê só as próprias entregas e atualiza o andamento.

Nome

E-mail

Perfil

Entregas

Acesso

Ana Administradora Você	admin@rotacerta.com.br	Administrador	Não se aplica	Ativo	Editar
Bruno Atendente	atendente@rotacerta.com.br	Atendente	Não se aplica	Ativo	EditarDesativar
Carla Entregadora	entregador@rotacerta.com.br	Entregador	Disponível Carga até 25 kg	Ativo	EditarDesativar
Diego Entregador	entregador2@rotacerta.com.br	Entregador	Disponível Carga até 40 kg	Ativo	EditarDesativar
Eduardo Nascimento	eduardo@rotacerta.com.br	Entregador	Disponível Carga até 30 kg	Ativo	EditarDesativar

Eduardo Nascimento cadastrado como entregador.

Figura 14 Entregador cadastrado e listado com disponibilidade e capacidade

18 Controle de perfis

Os três perfis previstos no RF01 foram implementados. A verificação acontece na API, em cada rota, e não apenas na interface: mesmo que alguém chame a API diretamente, a ação é negada para perfis sem permissão. O perfil é lido do banco a cada requisição, então uma alteração feita pelo administrador vale imediatamente.

Ação	Administrador	Atendente	Entregador
Entrar no sistema	Sim	Sim	Sim
Cadastrar, editar e desativar usuários	Sim	Não	Não
Consultar pedidos	Todos do estabelecimento	Todos do estabelecimento	Só os atribuídos a ele
Cadastrar e editar pedidos	Sim	Sim	Não
Atribuir entregador ao pedido	Sim	Sim	Não
Excluir pedidos	Sim	Não	Não
Atualizar a situação	Qualquer situação	Qualquer situação	De Atribuído para Em rota e de Em rota para Entregue

 **RotaCerta**
Farmácia Boa Saúde (demonstração)

Pedidos

Bruno Atendente
Atendente

Sair

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Todos 6

Pendente 2

Atribuído 2

Em rota 1

Entregue 1

Cancelado 0

Pedido

Cliente

Endereço de entrega

Horário

Prioridade

Entregador

Situação

#6

Paulo Mendes
(81) 98800-1006

Rua da Hora, 150 - Espinheiro, Recife - PE
1,5 kg

28/09, 14:00 até 16:00

Normal

Diego Entregador

Entregue

Editar

#5

Fernanda Lima
(81) 98800-1005

Rua Real da Torre, 900 - Madalena, Recife - PE
3 kg

28/09, 13:00 até 15:00

Alta

Sem entregador

Pendente

Editar

#4

Carlos Souza
(81) 98800-1004

Rua do Espinheiro, 450 - Espinheiro, Recife - PE
0,8 kg

28/09, 12:00 até 14:00

Baixa

Diego Entregador

Atribuído

Editar

#3

Ana Costa
(81) 98800-1003

Estrada do Encanamento, 800 - Casa Forte, Recife - PE
4 kg

28/09, 11:00 até 13:00

Normal

Sem entregador

Pendente

Editar

#2

João Pereira
(81) 98800-1002

Rua Amélia, 300 - Graças, Recife - PE
1,2 kg

28/09, 10:00 até 12:00

Normal

Carla Entregadora

Em rota

Editar

#1

Maria Silva
(81) 98800-1001

Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE
2,5 kg

28/09, 09:00 até 11:00

Alta

Carla Entregadora

Atribuído

Editar

Figura 15 Visão do atendente: sem a aba Usuários e sem o botão Excluir

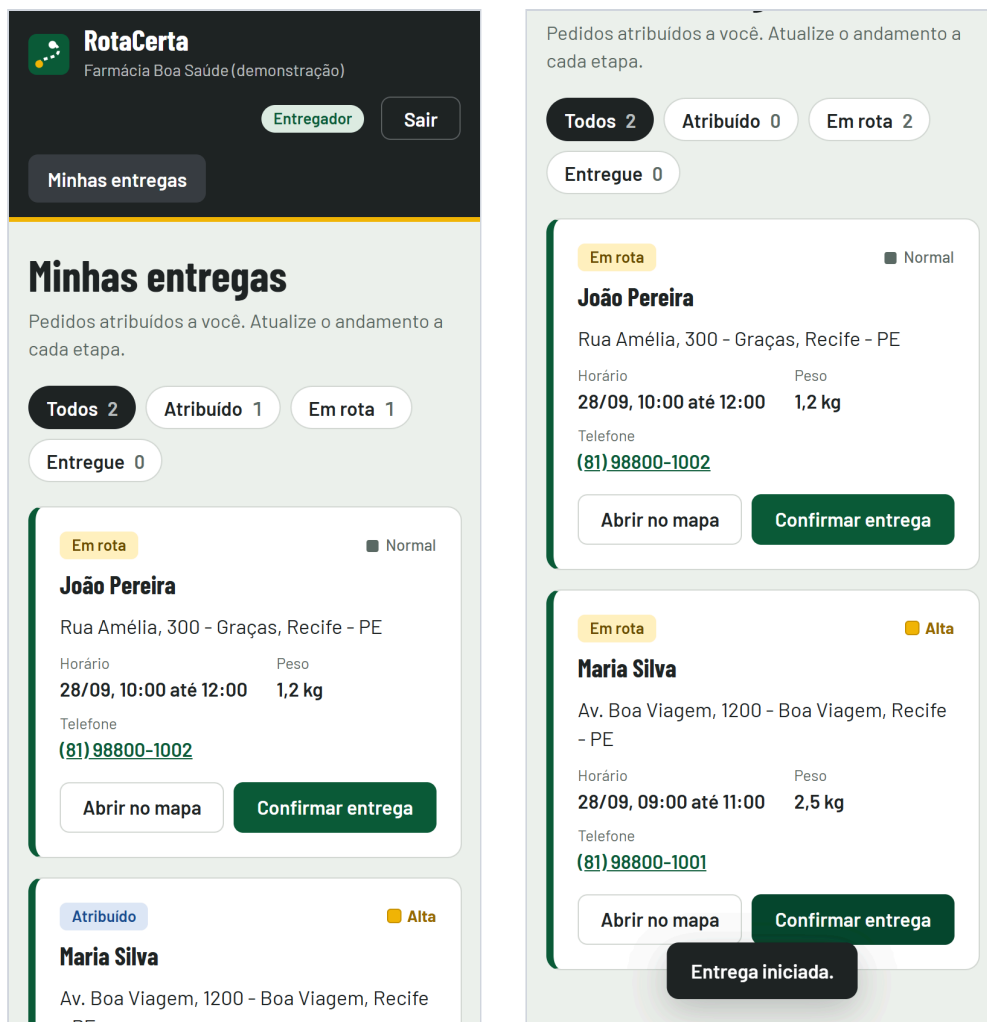


Figura 16 Visão do entregador no celular: apenas os próprios pedidos e o botão para avançar a entrega

```

GET /orders sem token
  -> HTTP 401 Sessão inválida ou expirada. Faça login novamente.
POST /auth/login com senha errada
  -> HTTP 401 E-mail ou senha inválidos.
POST /auth/login (Administrador)
  -> HTTP 200 token JWT emitido
POST /auth/login (Atendente)
  -> HTTP 200 token JWT emitido
POST /auth/login (Entregador)
  -> HTTP 200 token JWT emitido
GET /users como Administrador
  -> HTTP 200 5 registros
GET /users como Atendente
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
GET /users como Entregador
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
POST /orders como Entregador
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
GET /orders como Entregador
  -> HTTP 200 2 pedidos, todos atribuídos a ele
DELETE /orders/6 como Atendente
  -> HTTP 403 Seu perfil não tem permissão para esta ação.

```

Respostas da API para cada perfil, obtidas com chamadas diretas

19 CRUD principal: pedidos

O pedido é a entidade principal do RotaCerta porque concentra o que será roteirizado: cliente, destino, carga, prioridade, janela de horário, entregador e situação. O CRUD usa as tabelas orders e customers do modelo da Sprint 02. O cliente é informado no próprio formulário do pedido; a API reaproveita o cadastro quando o mesmo nome e telefone já existem e cria um registro em customers quando não existem. O local de entrega é marcado com um clique no mapa, que preenche latitude e longitude.

Operação	Rota da API	Regra de negócio
Cadastrar	POST /orders	Coordenadas obrigatórias, fim da janela posterior ao início; com entregador o pedido já nasce Atribuído
Consultar	GET /orders e GET /orders/{id}	Lista com filtro por situação; o entregador vê só os próprios pedidos
Atualizar	PUT /orders/{id} e PATCH /orders/{id}/status	Pedidos entregues ou cancelados não podem ser editados; Em rota exige entregador
Excluir	DELETE /orders/{id}	Somente administrador; pedido que já faz parte de uma rota é protegido pela chave estrangeira

 **RotaCerta**
Farmácia Boa Saúde (demonstração)

Pedidos

Usuários

Ana Administradora
Administrador

Sair

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Novo pedido

Todos 6Pendente 2Atribuído 2Em rota 1Entregue 1Cancelado 0

Pedido	Cliente	Endereço de entrega	Horário	Prioridade	Entregador	Situação		
#6	Paulo Mendes (81) 98800-1006	Rua da Hora, 150 - Espinheiro, Recife - PE 1,5 kg	28/09, 14:00 até 16:00	Normal	Diego Entregador	Entregue	Editar	Excluir
#5	Fernanda Lima (81) 98800-1005	Rua Real da Torre, 900 - Madalena, Recife - PE 3 kg	28/09, 13:00 até 15:00	Alta	Sem entregador	Pendente	Editar	Excluir
#4	Carlos Souza (81) 98800-1004	Rua do Espinheiro, 450 - Espinheiro, Recife - PE 0,8 kg	28/09, 12:00 até 14:00	Baixa	Diego Entregador	Atribuído	Editar	Excluir
#3	Ana Costa (81) 98800-1003	Estrada do Encanamento, 800 - Casa Forte, Recife - PE 4 kg	28/09, 11:00 até 13:00	Normal	Sem entregador	Pendente	Editar	Excluir
#2	João Pereira (81) 98800-1002	Rua Amélia, 300 - Graças, Recife - PE 1,2 kg	28/09, 10:00 até 12:00	Normal	Carla Entregadora	Em rota	Editar	Excluir
#1	Maria Silva (81) 98800-1001	Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE 2,5 kg	28/09, 09:00 até 11:00	Alta	Carla Entregadora	Atribuído	Editar	Excluir

Figura 17 Lista de pedidos do administrador com filtros por situação



RotaCerta
Farmácia Boa Saúde (demonstração)

Pedidos

Usuários

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Todos 6

Pendente 2

Atribuído 2

Em rota 1

Entregue 1

Cancelado 0

Pedido	Cliente	Endereço de entrega	Horário	Pr
#6	Paulo Mendes (81) 98800-1006	Rua da Hora, 150 - Espinheiro, Recife - PE 1,5 kg	28/09, 14:00 até 16:00	■
#5	Fernanda Lima (81) 98800-1005	Rua Real da Torre, 900 - Madalena, Recife - PE 3 kg	28/09, 13:00 até 15:00	■
#4	Carlos Souza (81) 98800-1004	Rua do Espinheiro, 450 - Espinheiro, Recife - PE 0,8 kg	28/09, 12:00 até 14:00	■
#3	Ana Costa (81) 98800-1003	Estrada do Encanamento, 800 - Casa Forte, Recife - PE 4 kg	28/09, 11:00 até 13:00	■
#2	João Pereira (81) 98800-1002	Rua Amélia, 300 - Graças, Recife - PE 1,2 kg	28/09, 10:00 até 12:00	■
#1	Maria Silva (81) 98800-1001	Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE 2,5 kg	28/09, 09:00 até 11:00	■

Novo pedido

Cliente

Nome

Luciana Barros

Telefone

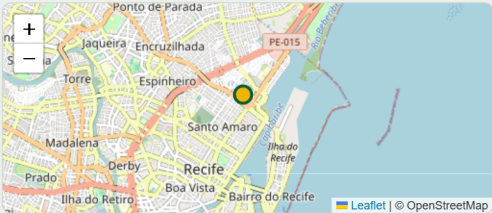
(81) 98800-2040

Entrega

Endereço

Rua das Graças, 210 - Graças, Recife - PE

Local no mapa



Clique no mapa para marcar o ponto de entrega.

Latitude

-8.045213

Longitude

-34.877815

Peso (kg)

2,5

Prioridade

Alta

Normal

Baixa

Entregar a partir de

28/09/2026 14:00

Entregar até

28/09/2026 16:00

Andamento

Entregador

Diego Entregador (disponível)

Cadastrar pedido

Figura 18 Cadastro de pedido com cliente, local no mapa, prioridade, janela e entregador

RotaCerta | Grupo 04 | Sprints 01 a 03 | Página 23

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Novo pedido

Todos 7 Pendente 2 Atribuído 3 Em rota 1 Entregue 1 Cancelado 0

Pedido	Cliente	Endereço de entrega	Horário	Prioridade	Entregador	Situação		
#7	Luciana Barros (81) 98800-2040	Rua das Graças, 210 - Graças, Recife - PE 2,5 kg	28/09, 14:00 até 16:00	Alta	Diego Entregador	Atribuído	Editar	Excluir
#6	Paulo Mendes (81) 98800-1006	Rua da Hora, 150 - Espinheiro, Recife - PE 1,5 kg	28/09, 14:00 até 16:00	Normal	Diego Entregador	Entregue	Editar	Excluir
#5	Fernanda Lima (81) 98800-1005	Rua Real da Torre, 900 - Madalena, Recife - PE 3 kg	28/09, 13:00 até 15:00	Alta	Sem entregador	Pendente	Editar	Excluir
#4	Carlos Souza (81) 98800-1004	Rua do Espinheiro, 450 - Espinheiro, Recife - PE 0,8 kg	28/09, 12:00 até 14:00	Baixa	Diego Entregador	Atribuído	Editar	Excluir
#3	Ana Costa (81) 98800-1003	Estrada do Encanamento, 800 - Casa Forte, Recife - PE 4 kg	28/09, 11:00 até 13:00	Normal	Sem entregador	Pendente	Editar	Excluir
#2	João Pereira (81) 98800-1002	Rua Amélia, 300 - Graças, Recife - PE 1,2 kg	28/09, 10:00 até 12:00	Normal	Carla Entregadora	Em rota	Editar	Excluir

Pedido #7 cadastrado.

Figura 19 Pedido cadastrado aparecendo no topo da lista


RotaCerta
Farmácia Boa Saúde (demonstração)

Pedidos
Usuários

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Todos 7
Pendente 2
Atribuído 3
Em rota 1
Entregue 1
Cancelado 0

Pedido	Cliente	Endereço de entrega	Horário	Pr
#7	Luciana Barros (81) 98800-2040	Rua das Graças, 210 - Graças, Recife - PE 2,5 kg	28/09, 14:00 até 16:00	
#6	Paulo Mendes (81) 98800-1006	Rua da Hora, 150 - Espinheiro, Recife - PE 1,5 kg	28/09, 14:00 até 16:00	
#5	Fernanda Lima (81) 98800-1005	Rua Real da Torre, 900 - Madalena, Recife - PE 3 kg	28/09, 13:00 até 15:00	
#4	Carlos Souza (81) 98800-1004	Rua do Espinheiro, 450 - Espinheiro, Recife - PE 0,8 kg	28/09, 12:00 até 14:00	
#3	Ana Costa (81) 98800-1003	Estrada do Encanamento, 800 - Casa Forte, Recife - PE 4 kg	28/09, 11:00 até 13:00	
#2	João Pereira (81) 98800-1002	Rua Amélia, 300 - Graças, Recife - PE 1,2 kg	28/09, 10:00 até 12:00	
#1	Maria Silva (81) 98800-1001	Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE 2,5 kg	28/09, 09:00 até 11:00	

Pedido #7 cadastrado.

Editar pedido #7

Cliente

Nome
Telefone

Luciana Barros
(81) 98800-2040

Entrega

Endereço

Rua das Graças, 210 - Graças, Recife - PE

Local no mapa



Clique no mapa para marcar o ponto de entrega.

Latitude
Longitude

-8.045213
-34.877815

Peso (kg)
Prioridade

2.5
Alta Normal Baixa

Entregar a partir de
Entregar até

28/09/2026 14:00
28/09/2026 17:30

Andamento

Entregador
Situação

Carla Entregadora (disponível)
Atribuído

Salvar alterações

Figura 20 Edição do pedido: prioridade, horário e entregador alterados

```
-- Pedido #7 gravado no PostgreSQL após cadastro e edição pela interface
SELECT o.id, c.full_name AS cliente, o.priority AS prioridade, o.status, u.full_name AS entregador, o.weight_kg AS
peso_kg, to_char(o.desired_end AT TIME ZONE 'America/Recife', 'DD/MM HH24:MI') AS entregar_ate FROM orders o JOIN
customers c ON c.id = o.customer_id LEFT JOIN couriers co ON co.id = o.assigned_courier_id LEFT JOIN users u ON u.id =
co.user_id WHERE o.id = 7
```

id	cliente	prioridade	status	entregador	peso_kg	entregar_ate
7	Luciana Barros	2	ASSIGNED	Carla Entregadora	2.50	28/09 17:30

(1 linha)

Pedido gravado no banco com os valores da edição

RotaCerta
Farmácia Boa Saúde (demonstração)

Pedidos
Usuários

Ana Administradora
Administrador
Sair

Pedidos

Cadastre os pedidos do dia e escolha quem vai entregar.

Todos 6

Pendente 2

Atribuído 2

Em rota 1

Entregue 1

Cancelado 0

Novo pedido

Pedido	Cliente	Endereço de entrega	Horário	Prioridade	Entregador	Situação		
#6	Paulo Mendes (81) 98800-1006	Rua da Hora, 150 - Espinheiro, Recife - PE 1,5 kg	28/09, 14:00 até 16:00	Normal	Diego Entregador	Entregue	Editar	Excluir
#5	Fernanda Lima (81) 98800-1005	Rua Real da Torre, 900 - Madalena, Recife - PE 3 kg	28/09, 13:00 até 15:00	Alta	Sem entregador	Pendente	Editar	Excluir
#4	Carlos Souza (81) 98800-1004	Rua do Espinheiro, 450 - Espinheiro, Recife - PE 0,8 kg	28/09, 12:00 até 14:00	Baixa	Diego Entregador	Atribuído	Editar	Excluir
#3	Ana Costa (81) 98800-1003	Estrada do Encanamento, 800 - Casa Forte, Recife - PE 4 kg	28/09, 11:00 até 13:00	Normal	Sem entregador	Pendente	Editar	Excluir
#2	João Pereira (81) 98800-1002	Rua Amélia, 300 - Graças, Recife - PE 1,2 kg	28/09, 10:00 até 12:00	Normal	Carla Entregadora	Em rota	Editar	Excluir
#1	Maria Silva (81) 98800-1001	Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE 2,5 kg		Alta	Carla Entregadora	Atribuído	Editar	Excluir

Pedido #7 excluído.

Figura 21 Pedido excluído pelo administrador

```
-- Consulta ao pedido #7 depois da exclusão pela interface
SELECT count(*) AS pedidos_encontrados FROM orders WHERE id = 7

pedidos_encontrados
-----
0
(1 linha)
```

O pedido excluído não existe mais no banco

20 Execução local

Com Docker

1. Instalar o Docker Desktop.
2. Na raiz do repositório, copiar .env.example para .env.
3. Executar docker compose up --build.
4. Se já existir um PostgreSQL usando a porta 5432, alterar POSTGRES_PORT no .env para 5433.

Sem Docker

1. Instalar Python 3.12 ou superior, Node.js 20 ou superior e PostgreSQL 16 ou superior.
2. No PostgreSQL, criar o usuário e o banco: CREATE ROLE rotacerta LOGIN PASSWORD 'rotacerta_dev'; CREATE DATABASE rotacerta OWNER rotacerta;
3. Na pasta backend, criar o ambiente virtual, instalar requirements.txt e executar uvicorn app.main:app --reload. Na primeira execução a API cria as tabelas e os dados de demonstração.
4. Na pasta frontend, executar npm install e npm run dev.

5. Para outra porta, usuário ou senha do banco, copiar backend/.env.example para backend/.env e ajustar DATABASE_URL.

Serviço	Endereço
Sistema	http://localhost:5173
Documentação interativa da API	http://localhost:8000/docs
Verificação da API e do banco	http://localhost:8000/health

Perfil	E-mail	Senha
Administrador	admin@rotacerta.com.br	rotacerta123
Atendente	atendente@rotacerta.com.br	rotacerta123
Entregador	entregador@rotacerta.com.br	rotacerta123

As contas acima são criadas automaticamente apenas quando o banco não possui nenhum estabelecimento, para facilitar a demonstração. O passo a passo completo também está no README do repositório.

21 Testes automatizados

Os testes da API usam um banco SQLite em memória, recriado a cada teste, e cobrem login, sessão, hash de senha, cadastro de usuários, isolamento entre estabelecimentos, permissões de cada perfil, CRUD de pedidos e regras de situação. Os testes do otimizador continuam validando os modos sequencial e paralelo.

```
$ python -m unittest discover backend/tests -v
test_admin_creates_user_who_can_log_in (test_api.ApiTestCase.test_admin_creates_user_who_can_log_in) ... ok
test_courier_advances_status_in_order (test_api.ApiTestCase.test_courier_advances_status_in_order) ... ok
test_courier_cannot_create_orders (test_api.ApiTestCase.test_courier_cannot_create_orders) ... ok
test_courier_requires_load_capacity (test_api.ApiTestCase.test_courier_requires_load_capacity) ... ok
test_courier_sees_only_own_orders (test_api.ApiTestCase.test_courier_sees_only_own_orders) ... ok
test_deactivated_user_cannot_log_in (test_api.ApiTestCase.test_deactivated_user_cannot_log_in) ... ok
test_delivered_order_cannot_be_edited (test_api.ApiTestCase.test_delivered_order_cannot_be_edited) ... ok
test_duplicate_email_is_rejected (test_api.ApiTestCase.test_duplicate_email_is_rejected) ... ok
test_login_rejects_wrong_password (test_api.ApiTestCase.test_login_rejects_wrong_password) ... ok
test_login_returns_token_and_profile (test_api.ApiTestCase.test_login_returns_token_and_profile) ... ok
test_only_admin_manages_users (test_api.ApiTestCase.test_only_admin_manages_users) ... ok
test_order_crud (test_api.ApiTestCase.test_order_crud) ... ok
test_order_window_must_be_valid (test_api.ApiTestCase.test_order_window_must_be_valid) ... ok
test_password_is_stored_as_hash (test_api.ApiTestCase.test_password_is_stored_as_hash) ... ok
test_protected_routes_require_token (test_api.ApiTestCase.test_protected_routes_require_token) ... ok
test_register_creates_isolated_establishment (test_api.ApiTestCase.test_register_creates_isolated_establishment) ... ok
test_empty_route_has_zero_distance (test_optimizer.RoutingTestCase.test_empty_route_has_zero_distance) ... ok
test_nearest_neighbor_keeps_every_stop_once (test_optimizer.RoutingTestCase.test_nearest_neighbor_keeps_every_stop_once) ... ok
test_optimize_route_returns_order_and_distance (test_optimizer.RoutingTestCase.test_optimize_route_returns_order_and_distance) ... ok
test_parallel_and_sequential_return_same_routes (test_optimizer.RoutingTestCase.test_parallel_and_sequential_return_same_routes) ... ok
test_two_opt_never_worsens_route (test_optimizer.RoutingTestCase.test_two_opt_never_worsens_route) ... ok
Ran 21 tests in 4.576s
OK
```

Execução da suíte de testes

22 Ajustes no planejamento e na modelagem

Modelagem

Nenhuma tabela, coluna ou restrição do modelo relacional da Sprint 02 precisou ser alterada. A implementação tomou três decisões que não mudam o modelo:

- O cliente é cadastrado a partir do formulário do pedido, sem uma tela própria nesta etapa, reaproveitando a tabela customers.
- Usuários são desativados em vez de excluídos, porque pedidos e rotas referenciam entregadores por chave estrangeira.
- As coordenadas do pedido são obtidas com um clique no mapa Leaflet; a busca automática pelo endereço fica para a próxima etapa.

Planejamento

A Sprint 03 da disciplina exigiu login, perfis, cadastro de usuários e o CRUD principal, que no cronograma da Sprint 02 estavam previstos para as Sprints 04 e 05. Esses itens foram antecipados. As etapas seguintes foram reorganizadas para manter a otimização sequencial e paralela e os experimentos antes do período de relatórios e da banca.

Sprint	Período	Entrega principal	Situação
1	03/09 a 05/09	Planejamento, requisitos e backlog	Concluída
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura	Concluída
3	20/09 a 26/09	Banco conectado, login, perfis, usuários e CRUD de pedidos	Concluída
4	27/09 a 03/10	Entregadores, disponibilidade, clientes e busca de endereço	Planejada
5	04/10 a 10/10	Atribuição de pedidos e visualização das rotas no mapa	Planejada
6	11/10 a 17/10	Vizinho mais próximo integrado aos pedidos do dia	Planejada
7	18/10 a 24/10	2-opt, persistência das rotas e linha de base	Planejada
8	25/10 a 31/10	Paralelização para múltiplos entregadores	Planejada
9	01/11 a 07/11	Experimentos sequencial versus paralelo	Planejada
10	08/11 a 14/11	Histórico, relatórios e painel do dia	Planejada
11	15/11 a 21/11	Testes, usabilidade e segurança	Planejada
12	22/11 a 28/11	Documentação e ajustes	Planejada
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca	Planejada

23 Repositório GitHub

Repositório oficial. <https://github.com/alvccpj/rota-certa>

Branch	Finalidade
master	Versão estável e entregas aprovadas
sprint/02-arquitetura-modelagem	Integração da Sprint 02

Branch	Finalidade
sprint/03-estrutura-funcional	Integração da Sprint 03
dev/alvaro, dev/arthur, dev/vinicius	Ambientes pessoais dos integrantes

Caminho	Novidade da Sprint 03
backend/app/routers/	Rotas de autenticação, usuários e pedidos
backend/app/models.py e database.py	Conexão e mapeamento do PostgreSQL
backend/tests/test_api.py	Testes da API
frontend/src/pages/	Telas de login, pedidos e usuários
docs/sprint-03/	Capturas de tela e saídas usadas como evidência
tools/documentation/	Captura das evidências e geração deste documento

24 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Como foi tratada
Docker indisponível em máquina de desenvolvimento	Execução sem Docker documentada; a API cria as tabelas a partir do schema.sql quando o banco está vazio
PostgreSQL já instalado ocupando a porta 5432	Porta do banco no Compose passou a ser configurável por POSTGRES_PORT
psycopg 3.2.9 sem pacote para o Python 3.14	Atualização para psycopg 3.3.6, compatível com Python 3.12 a 3.14
Pedidos exigem latitude e longitude	Mapa no formulário para marcar o ponto; busca automática pelo endereço planejada
Paralelização mais lenta que a versão sequencial em entradas pequenas	Criar processos custa mais que o cálculo com poucos pedidos; os experimentos usarão cenários de 50 a 500 pedidos

Próximos passos

- Gerenciar entregadores e disponibilidade em tela própria e buscar coordenadas a partir do endereço.
- Integrar o otimizador aos pedidos do dia e gravar rotas em routes e route_stops.
- Registrar as execuções sequenciais e paralelas em optimization_runs.
- Distribuir as tarefas entre os integrantes com pull requests revisados, registrando a contribuição de cada um no GitHub.

Situação da Sprint 03

Entrega obrigatória	Situação	Evidência
Banco de dados conectado	Concluído	Seção 15: /health, consulta às tabelas e documentação da API
Login funcional	Concluído	Seção 16: tela de login, erro de senha e hash no banco

Entrega obrigatória	Situação	Evidência
Cadastro de usuários	Concluído	Seção 17: cadastro de estabelecimento e de usuários
Controle de perfis	Concluído	Seção 18: matriz de permissões, telas por perfil e respostas 401 e 403
CRUD principal	Concluído	Seção 19: cadastro, consulta, edição e exclusão de pedidos no banco
Deploy local	Concluído	Seção 20: Docker Compose ou execução manual
Repositório atualizado	Concluído	Seção 23: branch da Sprint 03